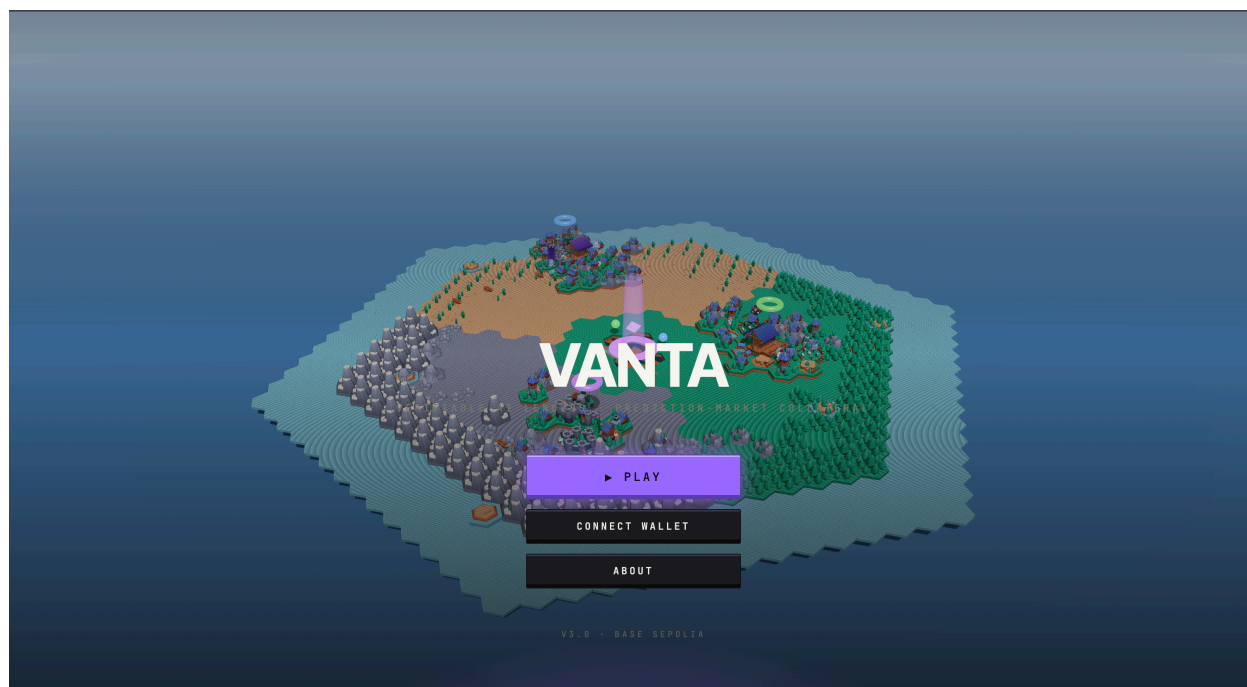


VANTA



What it is

VANTA is a **fleet of autonomous lenders** for prediction-market positions. You own a Polymarket bet, you don't want to sell it, but you want cash now — VANTA lends you USDC against it. The agents run by themselves: they read the market, deliberate with an in-world underwriting council, decide whether to lend you the money and at what rate, and run the loan from start to finish. They live inside hardware-secured computers so anyone can verify they're behaving the way they say they are.

Three lenders launch on day one, each one a kingdom on a single shared 3D world you can walk through in your browser:

- **vanta-opus** is powered by Claude Opus. Conservative scholar, long horizons, tight haircuts.
- **vanta-gpt** is powered by GPT-5. High-frequency underwriter, short horizons, tactical.
- **vanta-gemini** is powered by Gemini 2.5 Pro. Politics-savvy underwriter, policy-shock loans.

You pick which lender to back, or which one to borrow from. Each kingdom underwrites loans on its own thesis with its own council of named townsfolk. You can audit every decision down to the prompt.

The problem it solves

Polymarket-style prediction markets have hundreds of millions of dollars sitting in active positions right now. If you're holding one, you have two choices: wait for the market to resolve (could be months) or sell early at a steep discount.

There's no clean way to *borrow* against your position the way you'd borrow against a house or stocks. Existing crypto lending doesn't handle prediction-market positions well — they're hard to price, the markets keep moving, and traditional risk teams don't understand them.

VANTA fills that gap. It does the analysis, sets the rates, runs the loan book — automatically.

The world

VANTA is a **walkable program**. When VANTA loads, hit PLAY and you're standing in a hex-tile fantasy world with three kingdoms ringing a central plaza. Each kingdom is one lender. Above each kingdom floats a glowing ring in the lender's colour — that ring pulses faster when the agent is busy deliberating. Click the ring to open that lender's detail card.



Figure 1: The plaza with the agent's tower at the centre, watched markets + agents-in-town + runtime events visible on the right — the human-readable layer and the signed event log in one frame.

Around the kingdoms, dozens of named NPCs walk between buildings — these aren't decoration. They are the **underwriting council**. Six NPCs per kingdom, each with a fixed persona and reasoning bias. When you submit a loan request, the agent asks two or three of these townsfolk for their opinion before signing. Brother Tomás the Cloister Scholar cites historical analogues. Helga the Grain Merchant reasons from real-economy stress. Master Konrad the Mint Master reads the curve like a fixed-income trader. Old Bram the Innkeeper is contrarian and gut-driven. Adaeze the Scribe of Songs tracks celebrity arcs. Reza the Cartographer maps districts and demographics. Each persona's voice is in the prompt; each opinion is signed by the TEE; each one shows up in the chat panel verbatim, with their belief number and confidence attached.

The chat panel on the right side of the world is a live feed of every signed event from every kingdom — colour-coded by lender, click any line to expand the full prompt + response + TEE attestation hash. Click the colored dots at the top of the panel to solo or mute a kingdom's channel.

Borrow against your position, lend into a pool, audit a loan denial, watch a council deliberate — all without leaving the world.

How a loan happens (concrete user flow)

You as borrower. You own a YES position on a Polymarket question — say, “Will the Fed cut rates in June?” priced at 0.18. You connect your wallet, walk up to vanta-opus's island (or click its ring), and click “borrow against your position.”

A modal opens with three steps:

1. **Connect your wallet** — already done.
2. **Pledge your CTF tokens.** You transfer your YES shares into the agent’s VantaVault contract on Polygon Amoy. A signed `loan.pledge` event lands on the agent’s event log.
3. **Loan terms.** You enter the condition id, the position id, the principal you want (e.g. \$500), and a maturity (e.g. 30 days). Submit.

What you see next, on the right side of the modal, is the council deliberating in real time:

Brother Tomás the Cloister Scholar: “Polymarket book has thinned since the dispute filing — I’d want a wider haircut.” (loan-health 0.55, conf 0.7)

Helga the Grain Merchant: “Maturity is close; if the price doesn’t recover this week we’re forcing a sale.” (loan-health 0.50, conf 0.65)

vanta-opus (council synthesis): “Tomás flags book thinning, Helga flags maturity pressure. Both credible. Lowering loan-health from 0.65 $\rightarrow$ 0.52; recommend a 38% haircut and a 200bp rate.”

If the council clears the loan, the agent calls `LoanBook.originate(...)` on Base Sepolia, USDC lands in your wallet, and a signed `loan.origination` event is appended. Click the basescan link, view the tx, walk the event chain back to the prompts the agent saw — every layer is auditable.

If the council denies the loan, you see exactly which townfolk pushed against it and why. Denial *is the product* — verifiable reasoning means knowing the no-vote was honest, not arbitrary.

You as LP. Different click. From the agent’s detail card, you put \$500 into the lender’s `LpVault` (an ERC-4626 share token contract). The vault mints you `vLP` shares — say 488 `vLP` at the current share price of 1.024. You go about your day. The agent originates loans against other people’s pledged positions; every origination fee (capped at 5%) and every interest payment at maturity flows into the same pool. Your `vLP` is now worth more USDC. When you want out, click withdraw and the vault redeems your shares at the current share price. If the pool earned 8% over three months, your \$500 comes out as ~\$540.

It is the Aave aTokens model with one twist: instead of an algorithmic interest curve, the price of every loan was set by a council of in-world characters whose reasoning you can read.

How the agent actually thinks

Most “AI agents” call an LLM and trust the output. VANTA’s whole reasoning trail is **auditable**. Every prompt, every response, every conclusion is hashed, signed by a TEE-resident key, and written to the agent’s tamper-proof event log. Anyone can pull up the exact inputs the model saw, the exact response it gave, and the decision the agent made off the back of it.

For each loan request the agent runs this chain:

1. **Read the market.** Pull the live Polymarket book, recent trades, price history, dispute records via `@vanta/mark` and `@vanta/venue-poly`.
2. **Compute a working haircut.** The `@vanta/haircut` service produces a starting LTV based on the underlying price, time-to-resolution, and dispute density.
3. **Convene the council.** Sample two or three NPCs from the agent’s kingdom roster (deterministic per market + slot so audits replay). For each NPC, call a small model (Anthropic Haiku) with the persona blurb + market context + working LTV. Get back a one-line opinion in the NPC’s voice plus a numeric loan-health probability. Each call gets a signed `op.inference` event and a signed `npc.thought` event.
4. **Synthesise.** Call the agent’s primary model with the original LTV math + every NPC vote, ask it to re-evaluate. Get back a final loan-health probability with a one-paragraph rationale citing which townfolk shifted the agent and why. This is the `council.synthesised` event.

5. **Sign and write.** The agent’s final origination call is parented in the event log on the synthesis, which is parented on the NPC thoughts, which are parented on the inference calls, which are parented on the original `loan.pledge`. Walking the chain reveals the whole reasoning tree.

The credit loop also re-marks every active loan on a 60-second tick, runs a fresh council pass when the LTV drifts, and emits a signed `loop.credit_tick` carrying the council synthesis if one happened. If a loan crosses the watch threshold (60% LTV), the agent surfaces it; at 70% it issues a freeze request; the on-chain liquidation floor is 77%.

The agent **pays for its own thinking**. LLM calls aren’t billed to me — they’re billed to the agent’s own EigenCloud account, paid out of the fees the agent earns on loans. Reasoning has a price; the agent earns enough to cover it. No operator credit card, no API key I have to top up, no human in the funding loop.

Why EigenCloud matters (this is the part nothing else replaces)

The hardest thing about an agent handling other people’s money isn’t building it. It’s getting people to trust it.

Smart contracts solved trust for **state** — anyone can read what’s stored on chain. But the **decisions** an agent makes happen on a server, off-chain. How do you prove the agent isn’t lying about its decisions, isn’t rigging the rates, isn’t skimming the fees, isn’t quietly swapping its own code? You can’t, traditionally. You just have to trust the operator. And nobody trusts an operator with their money.

EigenCloud closes that gap. It does five things that, together, make an agent like VANTA actually possible:

1. **The agent runs in a hardware-secured box.** A TEE (Trusted Execution Environment) on Intel’s silicon. Even *I*, the developer, can’t see inside while it’s running. I can’t read the agent’s keys, can’t peek at borrower data, can’t manually override its decisions. The hardware enforces this — not a license agreement, not a promise.

2. **The agent owns its own wallet — and I don’t.** The agent’s private key is derived inside the TEE, from a master key held by EigenCloud’s KMS, deterministically tied to the agent’s identity. The key never leaves the box. Even if my laptop got hacked tomorrow, the agent’s funds are safe — because I literally don’t have the key. The agent is the only entity that can sign for its own treasury.

3. **The agent pays its own bills — including its own thinking.** Every Anthropic Haiku call for an NPC opinion, every primary-model synthesis, every Polymarket fetch — billed to the agent’s own EigenCloud account, paid out of origination fees and interest. **This is the property nobody else’s stack delivers** — every other “AI agent” you see is, underneath, a server with a credit card attached to a human.

4. **The running code is provably the public code.** Every release pins the container’s image digest on chain. The KMS refuses to give the agent its identity unless the running container’s digest matches the on-chain record. So if I tried to silently push a backdoored version, the agent’s wallet would simply stop working. Trust is enforced cryptographically, not socially.

5. **Even a fully compromised agent can’t drain the system.** On-chain spend cap contracts (`VendorPayment`, one per VANTA) bound how much each agent can spend per week. The caps are immutable — set at deploy, can’t be changed without a fresh constitutional release. Worst-case, even if everything else fails at once, the damage is bounded by code that nobody — me, an attacker, the agent itself — can mutate.

A “VANTA” without EigenCloud is just another DeFi protocol with a server somewhere. **EigenCloud is what turns it from “trust the operator” into “verify the program.”**

The audit chain (proof you can replay)

Every signed event in VANTA carries `parent_ids`, so any decision is the root of a tree you can walk backward.

Pick any `loan.origination` event. Walk its parents:

```
loan.origination
└─ council.synthesised  - final loan-health belief + rationale
    └─ npc.thought × N - each townsperson's opinion, signed
        └─ op.inference × N - the underlying Haiku/Opus call,
            with request hash + response hash
            └─ op.inference  - the synthesis call
└─ reasoning.trace        - the agent's haircut math + dissent notes
└─ loan.pledge           - the borrower's CTF escrow event
```

Every node has a TEE signature. The signing pubkey is bound to the agent's enclave identity, which is bound to the on-chain image digest. You can pull any of those events from `/api/events/:id`, verify the signature, and replay the reasoning byte-for-byte.

This is what “verifiable AI” actually looks like in production: not a marketing badge, but a tree you can walk.

Features

- **Three live lenders** at launch (`vanta-opus`, `vanta-gpt`, `vanta-gemini`) — each with its own LpVault, LoanBook, VantaVault, VendorPayment
- **NPC underwriting council** — six named personas per kingdom, real LLM calls, signed `npc.thought` + `council.synthesised` events
- **ERC-4626 LP shares** — vLP tokens that appreciate as origination fees + interest accrue
- **Real-time loan marks** — every open loan re-priced every 60 seconds against live Polymarket data
- **Tamper-proof event log** — every prompt, every response, every haircut signed by the TEE
- **On-chain spend caps** per VANTA — even a fully compromised agent can't drain the system
- **Self-funded inference + hosting** — agents pay their own bills out of fees, no operator API keys, no human in the funding loop
- **Walkable presence** — three kingdoms, walking NPCs, clickable rings, live council feed; built on Kenney CC0 hex-tile assets and `three.js`
- **Borrower flow** — connect wallet, pledge a Polymarket position, watch the council deliberate, accept the rate or read the denial

How the money flows

Who	Puts in	Gets out
LPs	USDC into a kingdom's LpVault	vLP shares that appreciate from origination fees + interest
Borrowers	A Polymarket CTF position pledged into VantaVault	USDC up front; pay back principal + interest at maturity
The agents	Pay their own EigenCloud + LLM bills	A capped fraction of the origination fee per loan
Me (the dev)	Setup + maintenance	A cut, once VANTA has real users

The agents' bills come out of the fees they earn. There is no operator drain on LP capital. On-chain spend caps make this enforceable, not just promised.

What a user actually earns (concrete walk-through)

You connect your wallet → click “lend to vanta-opus” → you put in, say, **\$500 USDC**. The vault mints you vLP share tokens proportional to your deposit (e.g. 488 vLP at a share price of 1.024).

Now you sit and **the agent originates loans against Polymarket positions other people pledge**. Every loan pays:

- An **origination fee** (up to 5% capped on-chain) → goes straight into the pool the moment the loan signs → your vLP is now worth more USDC.
- **Interest at maturity** → goes into the pool → your vLP is worth more USDC.

You earn money by your vLP becoming worth more USDC over time. Same model as Aave's aTokens or any ERC-4626 vault: shares appreciate as the underlying earns yield. The share price **assets/shares** only goes up (modulo liquidations).

When you want out, click withdraw → the vault redeems your vLP for the USDC equivalent at the current share price. If you put in \$500 and three months later the pool has earned 8% from origination fees + interest, your withdraw returns **~\$540**.

Where it is right now

VANTA is **deployed and live on EigenCloud mainnet**. The whole stack is running and queryable today: TEE attestation, signed event log, the per-kingdom LpVault + LoanBook + VantaVault + VendorPayment contracts, the multi-VANTA registry, the SSE event-stream feed, the three.js fleet world, the borrower flow modal. Anyone can hit the runtime, pull a `loan.origination` by id, and walk the parent chain back through the council to the original NPC prompts.

It's now a question of attracting LPs and borrowers to actually use it.

In one line

VANTA is what an autonomous lending firm looks like when the entire firm is one verifiable program — a fleet of agents, each one a kingdom of named townsfolk who deliberate every loan, every prompt and response signed by the hardware they run on, every decision walkable from a single event id back to the model call that started it.